

Q3 代码架构与运行逻辑完全指南

VLSI 布图规划设计 — 最小可行死区比例的二分搜索

2026 年第七届”华数杯”大学生数学建模竞赛 B 题

2026/08

目录

1	项目总体架构	2
1.1	目录结构	2
1.2	执行顺序	3
1.3	核心设计理念	3
2	Q3 求解流程	4
3	二分搜索算法 (q3_bisect.py)	5
3.1	判定调用 (-feas-only)	5
3.2	并行多种子判定	5
3.3	二分与终止	5
3.4	双向确认 (正确性关键)	5
4	Phase: d* 处完整求解 (q3_solver.py)	6
5	判定可靠性讨论	6
5.1	为什么需要并行多种子判定?	6
5.2	为什么需要双向确认?	6
5.3	与理论下界的关系	6
6	测试与验证	6
7	实际运行结果	7
7.1	最小死区比例 (并行判定 + 双向确认)	7
8	文件依赖关系	7

9 运行指南	7
9.1 环境要求	7
9.2 完整运行流程	8
9.3 常见问题	8
10 合规清单	8

1 项目总体架构

1.1 目录结构

```
1 The_7th_Huashu_Cup.../
2 |-- cpp_solver/                                # C++ 共享核心 (Q2 模式 + --feas-
   only 判定)
3 |   |-- src/
4 |       |-- floor_plan.hpp                    # B*-Tree 拓扑 + Skyline 解码 + 解析
   器
5 |       |-- sa.hpp                            # 两阶段退火 (run 找可行 + run2 精
   修)
6 |       |-- main.cpp                          # CLI (q2 模式 / --feas-only / --
   seed)
7 |-- common/
8 |   |-- verify.py                            # 独立合法性复核
9 |-- q3/                                        # 问题 3 客制层 (<-- 本文件)
10 |   |-- q3_bisect.py                         # 二分搜索 + 并行多种子判定 + 双向确
   认
11 |   |-- q3_solver.py                         # 驱动: 二分 + d* 处完整求解 + 汇总
   出图
12 |   |-- output/
13 |       |-- q3_metrics.csv                   # d*/side/HPWL/confirm 记录/合法性
14 |       |-- q3_<chip>_bisect.csv             # 二分轨迹 (可审计)
15 |       |-- q3_<chip>.rpt/log                # d* 处最优布局 + 收敛轨迹
16 |   |-- visualization/
17 |       |-- plot_bisect.py                   # 二分轨迹图 (区间收缩/可行标记/d*
   标注)
18 |       |-- plot_{floorplan,convergence,snapshots}.py
19 |       |-- figs/<时间戳>/<chip>/           # 每芯片 12 张图 (含 bisect 轨迹)
20 |-- paper/
```

1.2 执行顺序

执行顺序

1. `cd cpp_solver && make` (编译 bin/main)
2. `conda activate math`
3. `python q3/q3_solver.py --repeats 8` (二分 + d^* 处多起点求解 + 汇总 + 出图)

1.3 核心设计理念

问题本质：随死区比例 d 减小，轮廓 $W = H = \lceil \sqrt{\Sigma A(1+d)} \rceil$ 同步缩小，存在可行布局的概率单调下降——最小可行 d^* 是“可行性随 d 单调”的二分搜索问题。

判定可靠性：SA 是启发式，“找不到可行解”可能是搜索不足（假阴性）而非真不可行。为此： 每比例**并行多种子判定**（任一可行即可行）； 收敛后**双向确认**（ d^* 本身必须可行， $d^* - \varepsilon$ 处多种子必须全不可行）——保证 d^* 是“验证过的最小稳定点”。

2 Q3 求解流程

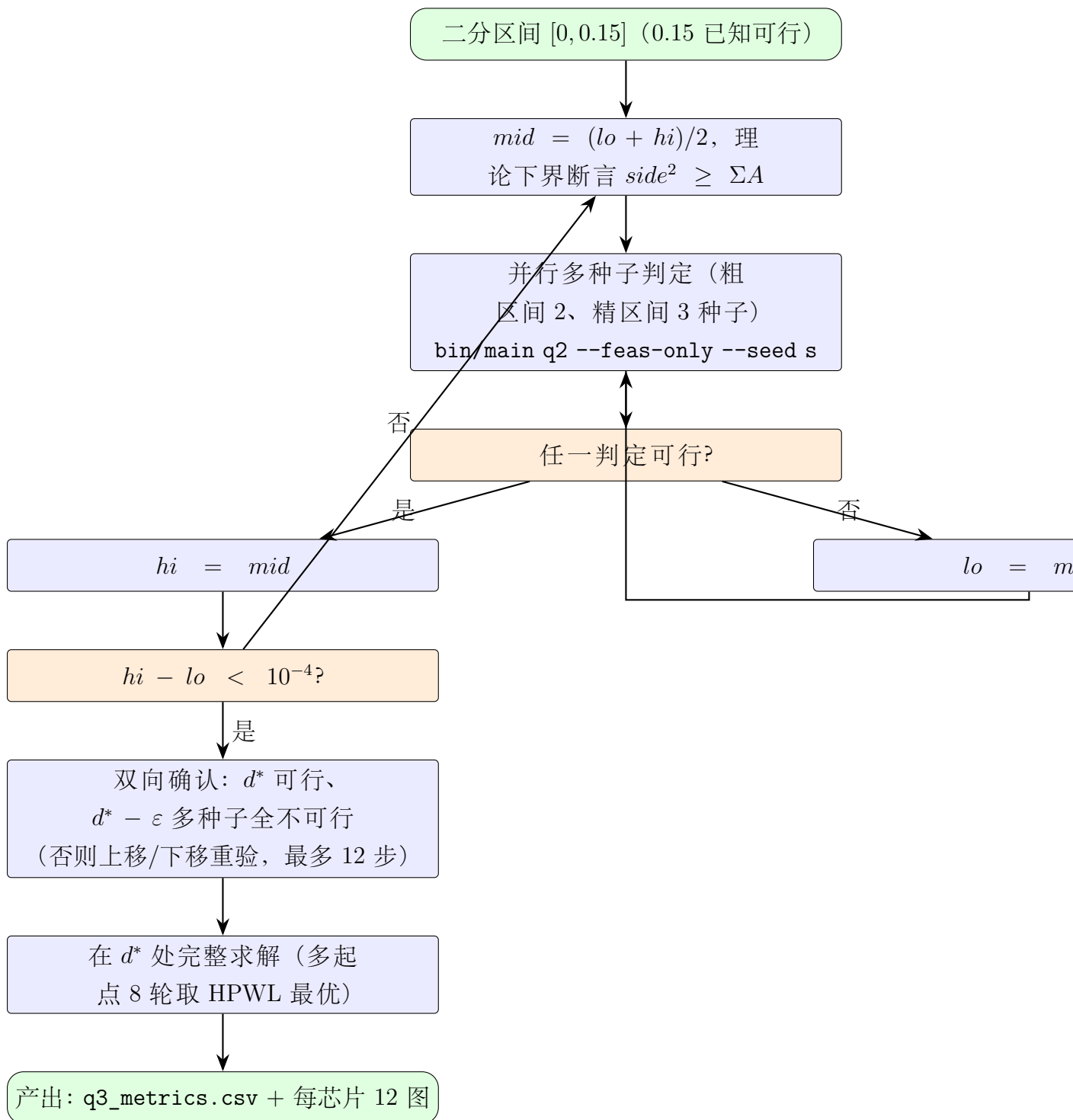


图 1: Q3 二分搜索 + 双向确认流程

3 二分搜索算法 (q3_bisect.py)

3.1 判定调用 (-feas-only)

共享核心提供可行性判定模式：

```
1 ./bin/main q2 0.5 <blocks> <nets> <pl> <rpt> <dead> --feas-only --  
   seed <s>  
2 # 输出 3 行: feasible(0/1) / bbox W H / hpwl
```

- 只跑 `run()` 可行性阶段，找到首个可行解即返回（早停）。
- `reset` 上限保证单次判定有界 ($N/16 + 1$ 次温度重置)。
- 固定种子 $\Rightarrow$ 判定可复现。

3.2 并行多种子判定

单种子判定存在假阴性（差种子搜索不足）。每比例并行 K 个独立种子（粗区间 $K = 2$ 、精区间 $K = 3$ ），任一可行即可行：

$$\text{feasible}(d) = \bigvee_{s=1}^K \text{feasible}_s(d), \quad P(\text{假阴性}) \approx \prod_{s=1}^K P(\text{假阴性}_s) \quad (1)$$

判定种子确定性与互异： `seed = BASE + chip_idx*10000 + tag*100 + i`。

3.3 二分与终止

- 区间 $[0, 0.15]$ (0.15 即 Q2 基准，已知可行)。
- 终止： $hi - lo < 10^{-4}$ (约 11 次迭代，精度 1.4×10^{-7} 内)。
- 每步记录轨迹(iter/lo/hi/mid/side/bbox/feasible/耗时)到 `q3_<chip>_bisect.csv` (可审计)。
- 理论下界全程断言： $side^2 \geq \Sigma A$ 恒成立。

3.4 双向确认（正确性关键）

二分收敛的 d_{bisect} 可能受判定波动影响，做双向确认：

1. 在 d 处做 $K_{\text{verify}} = 3$ 种子判定，**必须可行**；若不可行 $\Rightarrow$ 上移 $d += \varepsilon$ 重验。
2. 在 $d - \varepsilon$ 处做 3 种子判定，**必须全不可行**；若有可行 $\Rightarrow$ 下移 $d -= \varepsilon$ 重验。

3. 循环直至两点同时满足（最多 12 步），记录每步结果。

最终 d^* 满足: `d_feas=True` 且 `below_infeas=True` ($d^* - \varepsilon$ 处 `[False, False, False]`)
——即验证过的最小稳定点，写入 `confirm_steps` 可审计。

4 Phase: d^* 处完整求解 (q3_solver.py)

- 在 d^* 处调用完整 Q2 求解 (`run + run2 × 2`)，多起点 8 轮并行取 HPWL 最优（复用 q2 逻辑）。
- 只接受可行轮次 (`bbox ≤ side`)；保留 best 的 `rpt/log`。
- 独立复核：无重叠 / 尺寸保持 / 不越界 (`common/verify.py`)。
- 产出: `q3_metrics.csv` ($d^*/side/HPWL/confirm/合法性$) + 每芯片 12 张图 (9 快照 + 收敛 + 布图 + `bisect` 轨迹)。

5 判定可靠性讨论

5.1 为什么需要并行多种子判定?

早期版本（单种子判定）二分结果受种子运气影响：同一比例不同种子判定结果可相互矛盾（0.14209 判不可行、0.14206 判可行），导致 d^* 被推高（n300 达 0.142）。并行多种子判定将假阴性概率按种子数指数下降，n300 d^* 从 0.142 改善到 0.125。

5.2 为什么需要双向确认?

二分只保证”最后一个判可行的 `mid` 为上界”，不保证 $d^* - \varepsilon$ 处真不可行。双向确认把”可行/不可行”两个方向都验证到稳定， d^* 的语义是”在当前搜索强度下验证过的最小可行比例”——论文表述严谨（不声称超越搜索能力的真最优）。

5.3 与理论下界的关系

d 的理论下界为 0 ($side^2 ≥ \Sigma A$ 恒满足)；实际最小 d 由打包效率决定。搜索能力内的 d^* 是保守估计（可能略高于真值），但确认记录保证其”验证过”性质。

6 测试与验证

- 理论下界断言：每次判定 $side^2 ≥ \Sigma A$ 。
- **confirm 记录**: `d_feas` 与 `below_infeas` 双条件满足才算通过(`d_confirmed_minimum=True`)。

- 轨迹审计: `q3_<chip>_bisect.csv` 每步完整记录。
- 最终布局复核: Python独立验证 (无重叠/尺寸/越界)。
- C++ 测试: `feas-only` 正反例、`run` 快照、HPWL oracle (1135 断言)。

7 实际运行结果

7.1 最小死区比例 (并行判定 + 双向确认)

芯片	d^*	side	HPWL	迭代	确认步数	确认通过	合法性
n100	0.117307	448	293671	11	2	是	合法
n200	0.108977	442	544534	11	4	是	合法
n300	0.124885	555	826685	11	6	是	合法

- 全部芯片 `d_confirmed_minimum=True`: 最终 $d^* - \varepsilon$ 处 3 种子判定全不可行 (`below_results=[False]`)
- n300 确认过程含 4 次上移 (d^* 处判定波动), 最终在 0.124885 稳定——确认记录完整可审计。
- d^* 处 HPWL 大于 Q2 (轮廓更紧、搜索更困难), 合法且合理。

8 文件依赖关系

```

1 q3_solver.py
2 |-- q3_bisect.py                (二分 + 判定 + 确认)
3 |-- cpp_solver/bin/main        (q2 模式 + --feas-only + --seed)
4 |-- data/raw/{n100,n200,n300}.{blocks,nets,pl}
5 |-- common/verify.py
6 |-- q3/visualization/plot_{bisect,floorplan,convergence,snapshots}.
   py

```

9 运行指南

9.1 环境要求

- g++ (`-std=c++17`); conda 环境 `math`。
- TeX Live 2026 (`xelatex + ctex`, 编译本指南)。

9.2 完整运行流程

```
1 cd cpp_solver && make
2 cd ..
3 conda activate math
4 python q3/q3_solver.py --repeats 8 # 二分 + d* 求解 + 汇总 + 出图
```

9.3 常见问题

1. **d* 偏保守**: 判定强度与搜索深度相关——可提高判定种子数 (PRECISE_SEEDS) 或判定搜索强度 (参数优化阶段)。
2. **确认失败**: `d_confirmed_minimum=False` 时结果不可交付——增大确认步数或判定种子后重跑。
3. **复现**: 所有判定/求解使用固定种子公式 (`BASE + chip_idx*10000 + tag*100 + i`), 可精确复现。

10 合规清单

- 原始数据只读; 二分/判定/求解全部种子显式管理 (可复现)。
- `d_confirmed_minimum` 为交付硬性前提 (三芯片均为 True)。
- 判定波动与保守性如实记录 (第 5 节), 未作任何美化。
- 论文表述建议: ”在当前搜索强度下验证过的最小可行死区比例”。